
EagleEye

Cloud Architecture & Technology Stack

A Detailed Report on the Technologies Used, Their Roles, and Their Costs

EagleEye is a smart AI surveillance system built around an ESP32-CAM. This report explains the migration from a local (LAN) architecture to a fully cloud-connected, remote-capable architecture, and documents every technology in the new stack — what it does, why it was chosen, and whether it is free or paid.

Project	EagleEye — AI Surveillance System
Core device	ESP32-CAM (AI Thinker)
Document	Cloud Technology Stack Report
Scope	Architecture, technologies, pricing, comparison

Pricing figures reflect publicly advertised free/paid tiers at the time of writing and should be re-verified before deployment.

Contents

1	Introduction	2
1.1	The architectural shift	2
1.2	What this report covers	2
2	Architecture Overview	2
2.1	Old architecture (local / LAN)	2
2.2	New architecture (cloud)	2
3	The Technology Stack at a Glance	3
4	Technologies in Detail	3
4.1	MQTT (Message Queuing Telemetry Transport)	3
4.2	HiveMQ Cloud	4
4.3	TLS / Let's Encrypt	4
4.4	Cloudinary	4
4.5	Firebase Realtime Database	5
4.6	Deno & Deno Deploy	5
4.7	WebSocket Protocol	5
4.8	React Native & Expo	6
4.9	mqtt.js	6
4.10	ESP32-CAM & the Arduino Framework	6
4.11	Edge Impulse (On-device AI)	7
4.12	Supporting ESP32 Arduino Libraries	7
5	Cost Analysis: Free vs Paid	7
6	Old vs New: Summary of Changes	9
7	Conclusion	9

1 Introduction

EagleEye is an embedded AI surveillance node: an ESP32-CAM runs an on-device neural network that detects humans, captures an image of an intruder, drives a servo to pan the camera, and streams live video on request. The original prototype worked only on a **single local Wi-Fi network** and depended on a PC. This report documents the redesign into a **cloud-connected product** where the camera can be at one site and the phone anywhere in the world.

1.1 The architectural shift

The single most important change is the **direction of communication**. In the old design the phone had to reach *into* the camera using its local IP address. In the new design the camera connects *outward* to cloud services and stays connected — exactly how a messaging app receives messages on a phone anywhere without anyone knowing the phone’s IP. Because the device initiates every connection, there is no need for a fixed IP, port forwarding, or a PC on site, and the system works across any internet connection (home Wi-Fi, office, or 4G/LTE).

1.2 What this report covers

Section 2 gives a high-level before/after picture. Section 3 lists the full stack at a glance. Section 4 documents **each technology** in detail with its role and pricing. Section 5 summarises the total cost. Section 6 compares old vs new, and Section 7 concludes.

2 Architecture Overview

2.1 Old architecture (local / LAN)

```

SITE (one local Wi-Fi)
ESP32-CAM + Helper-ESP32(servo) --local MQTT--> PC (Mosquitto + bridge.py)
                                           |--> Cloudinary (image)
                                           |--> Firebase (record)
Phone --- same Wi-Fi, types camera IP ---> ws://<ip>:81 (video), http://<ip>/servo

```

Problems: works only on the same Wi-Fi; the camera IP must be typed and changes often; a PC must stay powered to run the broker and the upload bridge; a second ESP32 board is needed just for the servo; the local broker is unencrypted and open.

2.2 New architecture (cloud)

SITE A	CLOUD	ANYWHERE
ESP32-CAM (AI + servo) --TLS-->	HiveMQ broker <--wss--	Phone app (control)
image --HTTPS-->	Cloudinary	
alert --REST-->	Firebase	Phone app (history)
video --wss-->	Deno relay	Phone app (live, on-demand)

Result: no PC, no helper board, no LAN IP. The camera reaches out to the cloud; the phone reaches the same cloud; they meet in the middle. It works from any network and is encrypted end-to-end.

3 The Technology Stack at a Glance

Technology	Role in EagleEye	Cost
MQTT (protocol)	Lightweight messaging for commands/status/alerts	FREE
HiveMQ Cloud	Managed MQTT broker (the cloud "post office")	FREE tier / PAID upgrades
TLS / Let's Encrypt	Encryption + certificates for all connections	FREE
Cloudinary	Stores intruder images, returns a URL	FREE tier / PAID upgrades
Firebase Realtime DB	Stores the alert history the app reads	FREE tier / PAID upgrades
Deno & Deno Deploy	Hosts the live-video relay server	FREE tier / PAID upgrades
WebSocket (protocol)	Transport for relayed video + app↔broker	FREE
React Native + Expo	The cross-platform mobile app	FREE tier / PAID upgrades
mqtt.js	MQTT client library inside the app	FREE
ESP32-CAM + Arduino	The camera hardware + firmware framework	hardware only
Edge Impulse	Trains / exports the human-detection AI model	FREE tier / PAID upgrades
ESP32 Arduino libs	PubSubClient, ArduinoJson, WebSockets, ESP32Servo	FREE

4 Technologies in Detail

4.1 MQTT (Message Queuing Telemetry Transport)

Type: Open messaging protocol / standard

Cost: **FREE**(the protocol is royalty-free)

What it is. MQTT is a lightweight publish/subscribe messaging protocol designed for IoT devices with limited bandwidth and power. Devices connect to a central *broker* and exchange small messages on named *topics* (mailboxes). A device can *publish* to a topic and/or *subscribe* to receive everything posted there.

Role in EagleEye. MQTT is the control backbone. The camera publishes its **status** and **alert** topics and subscribes to a **cmd** topic; the app publishes commands (arm/disarm, servo angle, start/stop stream). Two features are especially useful: *retained messages* (the broker keeps the last status so the app sees current state instantly) and the *Last Will and Testament* (the broker auto-announces the camera is offline if it disconnects unexpectedly).

Free or paid? The protocol itself is an open OASIS/ISO standard — completely **FREE**. Costs only arise from the broker that hosts it (next subsection).

Why chosen. It is the de-facto industry standard for IoT command-and-control (used by AWS IoT, Azure IoT Hub, etc.), extremely light on the ESP32, and naturally outbound-connecting.

4.2 HiveMQ Cloud

Type: Managed cloud MQTT broker (SaaS)

Cost: **FREE** tier / **PAID** upgrades

What it is. HiveMQ Cloud is a hosted, always-on MQTT broker. Instead of running our own broker on a PC, we use HiveMQ's managed service, which provides a secure endpoint, user credentials, and per-topic access control.

Role in EagleEye. It is the cloud "post office" both the camera and the app connect to. The camera connects over TLS (port 8883); the app connects over secure WebSocket (port 8884). All control, status, and alert signalling flows through it.

Free or paid? HiveMQ Cloud offers a **FREE** Serverless tier (sufficient for this project — it supports a small number of devices and a monthly data allowance at no cost and requires no credit card). **PAID** plans (Starter / Professional / dedicated clusters) scale to more connections, more data throughput, and enterprise features. EagleEye runs entirely on the free tier.

Why chosen. Zero-maintenance, secure by default (TLS + per-credential access), free for our scale, and removes the PC-hosted broker entirely.

4.3 TLS / Let's Encrypt

Type: Transport-layer encryption + certificate authority

Cost: **FREE**

What it is. TLS (Transport Layer Security) is the same encryption that powers `https://`. It encrypts data in transit so no one on the network can read or tamper with it. Certificates (often issued free by Let's Encrypt) let the client verify it is talking to the genuine server.

Role in EagleEye. Every cloud connection is encrypted: MQTT over TLS to HiveMQ, HTTPS to Cloudinary and Firebase, and secure WebSocket (**wss**) to the relay. This replaces the old open, unencrypted local broker.

Free or paid? **FREE**— TLS is built into the libraries, and certificates from Let's Encrypt (used by the cloud providers) are free.

Why chosen. Security is mandatory once data leaves the local network; TLS is the universal standard and is provided automatically by every service we use.

4.4 Cloudinary

Type: Cloud image storage & delivery (media CDN)

Cost: **FREE** tier / **PAID** upgrades

What it is. Cloudinary is a media-management cloud that stores images and serves them over a fast global URL. It supports *unsigned upload presets*, which let a device upload directly without holding a secret key.

Role in EagleEye. When the AI detects an intruder, the camera uploads the captured JPEG *directly* to Cloudinary over HTTPS and receives back a public image URL. That URL is then stored in the alert record. This removed the PC bridge that used to do the upload.

Free or paid? Cloudinary has a **FREE** plan with a monthly allowance (storage, bandwidth, and transformations measured in "credits"), which is ample for a surveillance demo. **PAID** plans add

more storage/bandwidth and advanced features. EagleEye uses the free plan with an unsigned upload preset scoped to one folder.

Why chosen. Free, gives an instant shareable URL the app can display, and supports secure device-side uploads without embedding secret keys in the firmware.

4.5 Firebase Realtime Database

Type: Cloud NoSQL real-time database (Google)

Cost: **FREE** tier / **PAID** upgrades

What it is. Firebase Realtime Database (RTDB) is a cloud-hosted JSON database that pushes updates to connected clients in real time. When data changes, every listening app updates instantly.

Role in EagleEye. It stores the **alert history**. The camera writes each alert (**timestamp**, **image_url**, **type**) into the **alerts** node via the REST API; the mobile app listens to that node and shows new intrusions immediately in the dashboard and gallery.

Free or paid? The **FREESpark** plan covers our needs (a generous storage and download allowance and a limited number of simultaneous connections). The **PAIDBlaze** (pay-as-you-go) plan is only needed for higher volume or for Cloud Functions / push notifications. EagleEye's alert history runs on the free tier.

Why chosen. Real-time updates with no server code, an easy REST API the ESP32 can call directly, and it was already integrated in the original app, so the alert UI needed no changes.

4.6 Deno & Deno Deploy

Type: JavaScript/TypeScript runtime + serverless edge hosting

Cost: **FREE** tier / **PAID** upgrades

What it is. Deno is a modern, secure runtime for JavaScript/TypeScript (an alternative to Node.js). *Deno Deploy* is its serverless hosting platform that runs code on edge servers worldwide with built-in HTTPS/WebSocket support.

Role in EagleEye. It hosts the **live-video relay** — a small program that acts as a "meeting room": the camera connects out and pushes JPEG frames; the phone connects out and receives them. The relay simply forwards frames, so neither side needs the other's IP. This is what makes remote live video possible.

Free or paid? Deno Deploy has a **FREE** tier with a monthly request and bandwidth allowance that comfortably covers on-demand surveillance streaming. **PAID**(Pro) plans raise those limits. The relay runs on the free tier.

Why chosen. Free, always-on, supports WebSockets natively, deploys a single file in minutes, and terminates TLS automatically (giving a **wss://** URL with no certificate work).

4.7 WebSocket Protocol

Type: Full-duplex persistent network protocol / standard

Cost: **FREE**

What it is. WebSocket is a protocol that keeps a single connection open for continuous two-way data, unlike normal HTTP request/response. It is ideal for streaming and live updates.

Role in EagleEye. It carries the **live video frames** between the camera, the relay, and the app, and also carries MQTT messages between the app and HiveMQ (MQTT-over-WebSocket). Persistent connections mean low latency and no constant polling.

Free or paid? **FREE**— an open web standard built into browsers, React Native, and the ESP32 libraries.

Why chosen. It is the natural transport for continuous binary frames and is supported everywhere (device, relay, and phone), letting the same decode/render path be reused.

4.8 React Native & Expo

Type: Cross-platform mobile app framework + tooling

Cost: **FREE** tier / **PAID** upgrades

What it is. React Native is an open-source framework (by Meta) for building native Android and iOS apps from a single JavaScript codebase. Expo is a layer on top that simplifies development, testing (via the Expo Go app), and building.

Role in EagleEye. It is the entire **mobile app**: the dashboard (arm/disarm, online status, latest alert), the live-view screen with the servo pan slider, the gallery, and the statistics charts. One codebase runs on both Android and iOS.

Free or paid? React Native and the Expo SDK are open-source and **FREE**. Testing in **Expo Go** is free. Expo's **EAS** cloud build/submit/push service has a **FREE** tier (a limited number of builds per month) with **PAID** plans for higher build volume and priority. EagleEye is developed and demoed for free in Expo Go.

Why chosen. One codebase for both platforms, fast iteration with live reload, large ecosystem, and a free path from development to a shippable build.

4.9 mqtt.js

Type: Open-source MQTT client library (JavaScript)

Cost: **FREE**

What it is. `mqtt.js` is the most widely used JavaScript MQTT client. It runs in browsers and React Native and speaks MQTT over WebSocket.

Role in EagleEye. It is the app's connection to HiveMQ: it publishes commands (arm, servo, stream) and subscribes to the camera's status and stream-ready signals. It replaced the old "type the camera's IP" approach for control.

Free or paid? **FREE**— open-source (MIT/EPL).

Why chosen. Pure JavaScript (works in Expo Go without native modules), maintained, and the standard choice for MQTT in web/mobile apps.

4.10 ESP32-CAM & the Arduino Framework

Type: Microcontroller-with-camera hardware + open firmware framework

Cost: Hardware cost only; software **FREE**

What it is. The ESP32-CAM (AI Thinker) is a low-cost board combining a dual-core ESP32 (Wi-Fi + processing) with an OV2640 camera and PSRAM. It is programmed with the open-source ESP32 Arduino core.

Role in EagleEye. It is the **device itself** — it runs the AI, captures images, drives the servo on GPIO 15, connects to the cloud, and streams video. In the new design it also took over the servo job (no second board).

Free or paid? The **hardware** is a one-time purchase (roughly \$6–\$10 per board, plus a USB programmer and a 5V supply). The **software** — the Arduino core and all toolchains — is **FREE** and open-source.

Why chosen. Extremely low cost for a Wi-Fi camera with enough power to run a small neural network on-device, with a huge community and free tooling.

4.11 Edge Impulse (On-device AI)

Type: TinyML model training & deployment platform

Cost: **FREE** tier / **PAID** upgrades

What it is. Edge Impulse is a platform for collecting data, training small neural networks, and exporting them as optimised C++ libraries that run on microcontrollers (TinyML).

Role in EagleEye. It produced the **human-detection model** (an RGB 96×96 classifier) that runs *on the ESP32 itself* — no cloud inference, no per-image AI cost. The exported library performs the resize, normalisation, and neural-network inference on-device every frame.

Free or paid? Edge Impulse has a **FREE Developer/Community** plan that covers training and exporting models for personal/educational projects. **PAID Enterprise** plans add collaboration, larger datasets, and commercial features. The exported model runs on-device for free forever. EagleEye uses the free plan.

Why chosen. It makes on-device AI accessible without deep ML expertise, and on-device inference means privacy (images analysed locally) and zero ongoing AI cost.

4.12 Supporting ESP32 Arduino Libraries

Type: Open-source firmware libraries

Cost: **FREE**

These libraries (all **FREE**, open-source) implement the device side of each technology:

- **PubSubClient** — the MQTT client on the ESP32 (publishes status/alerts, receives commands), used over a TLS connection.
- **ArduinoJson** — builds and parses the JSON messages for commands, status, and alert records.
- **WebSockets (by Markus Sattler)** — the WebSocket client that pushes video frames to the Deno relay.
- **ESP32Servo** — generates the servo control signal on GPIO 15, using a timer that does not conflict with the camera.
- **esp32-camera** — the camera driver (capture + JPEG conversion).

Why chosen. They are the standard, well-maintained, free building blocks of the ESP32 Arduino ecosystem and map one-to-one onto the cloud services above.

5 Cost Analysis: Free vs Paid

A central design goal was that the entire cloud stack runs on **free tiers**, so the only real cost is the hardware. The table below summarises each item.

Item	Free tier used by EagleEye	When you would pay
HiveMQ Cloud	Serverless tier (small fleet, monthly data)	Many devices / high message volume
Cloudinary	Free plan (monthly credit allowance)	Large media storage / bandwidth
Firebase RTDB	Spark plan	High traffic, Cloud Functions, push
Deno Deploy	Free tier (requests + bandwidth)	Heavy / continuous streaming
Expo (EAS)	Expo Go + free build allowance	Frequent production builds
Edge Impulse	Developer/Community plan	Open/Commercial/enterprise use
MQTT, TLS, WebSocket, RN, mqtt.js, Arduino libs	Open standards & open-source	Never (always free)
Hardware	ESP32-CAM + programmer + 5 V supply	One-time purchase (≈\$15–\$25 total)

Bottom line. For a final-year-project scale (one camera, one or two users), the **recurring software cost is \$0/month**. The only spend is the one-time hardware. The architecture scales to paid tiers smoothly if it were ever commercialised, without changing the design.

6 Old vs New: Summary of Changes

Aspect	Old (LAN)	New (Cloud)
Reach	Same Wi-Fi only	Anywhere via the internet
Addressing	Type the camera's IP	Device ID; no IP needed
Broker	Mosquitto on a PC, no TLS	HiveMQ Cloud, TLS-secured
Image upload	PC bridge (<code>bridge.py</code>)	Camera uploads directly to Cloudinary
Alert storage	Written by the PC bridge	Camera writes Firebase directly
Live video	Direct to camera IP	Cloud relay, on-demand
Control	Firebase + bridge relay	Direct MQTT commands
Online status	Heartbeat guesswork	MQTT retained status + Last Will
Servo	Separate "helper" ESP32	Driven on the main board (GPIO 15)
PC required	Yes	No
Security	Open, unencrypted	Encrypted + per-device credentials

7 Conclusion

The redesign of EagleEye replaced a fragile, location-locked, PC-dependent system with a clean, secure, cloud-connected one. The guiding principle — *the device reaches out to the cloud instead of waiting to be reached* — eliminated the manual-IP problem, the on-site PC, and the extra microcontroller, while adding encryption, a truthful online/offline indicator, push-style alerting, and the ability to operate from anywhere.

Equally important for a student project, every cloud technology in the stack was selected to run on a **free tier**: MQTT, TLS, and WebSocket are open standards; HiveMQ, Cloudinary, Firebase, Deno Deploy, Expo, and Edge Impulse all provide capable free plans; and the firmware libraries are open-source. The result is a professional, industry-aligned architecture (mirroring how commercial IoT platforms such as AWS IoT and Ring/Nest cameras are built) at **zero recurring software cost**, with a clear, unchanged path to paid tiers should the system ever be scaled to many cameras and users.

EagleEye FYP — Cloud Architecture & Technology Stack report. Free/paid tiers are based on publicly advertised pricing at the time of writing; verify current limits with each provider before relying on them.